

Kitchen Editor Object JSON Creation Guideline

Updated single guideline for creating raw object JSON definitions in the Kitchen Editor project.

This guideline combines the object JSON creation rules and the cabinet rail/panel overlap rules into one reference. It is written for AI/code assistants that need to add or edit JSON object definitions without introducing object-specific TypeScript builders or unsupported JSON behavior.

1. Current project base

Use the current Kitchen Editor codebase with the raw JSON assembly engine.

Objects are defined as JSON files under:

```
src/features/kitchen-editor/catalogs/data/
```

The project uses one universal object format for basic units, doors, drawers, boxes, panels, and cabinets. The JSON file describes the assembled object. The generic raw assembly builder reads the JSON and creates the object.

Do not create object-specific TypeScript builders for new object definitions.

Use TypeScript changes only when the generic engine is missing a reusable capability. If the requested object needs unsupported behavior, explain the missing generic capability instead of inventing unsupported JSON fields.

2. How to add a new object

To add a new object:

1. Create a new JSON file in the correct catalog data folder.
2. Import that JSON file in the raw catalog registry:

```
src/features/kitchen-editor/catalogs/registry/kitchenEditorRawCatalogEntries.ts
```

3. Add one registry entry with:

```
catalogId  
categoryId  
rawDefinition
```

Example:

```
import baseThreeDoorCabinetRawDefinition from "../data/cabinets/base-cabinets/base-three-door-cabinet.json";  
  
export const kitchenEditorRawCatalogEntries = [  
  {  
    catalogId: "cabinets",  
    categoryId: "base-cabinets",  
    rawDefinition: baseThreeDoorCabinetRawDefinition,  
  },  
] as const satisfies readonly KitchenEditorRawCatalogEntry[];
```

Do not edit the catalog UI unless adding a completely new catalog or category.

3. Current catalog folders and IDs

Use these existing folders and IDs.

Basic Units

```
catalogId: "basic-units"
```

Categories:

```
categoryId: "boxes"
```

```
categoryId: "doors"
```

```
categoryId: "drawers"
```

```
categoryId: "panels"
```

Folders:

```
src/features/kitchen-editor/catalogs/data/basic-units/boxes/
```

```
src/features/kitchen-editor/catalogs/data/basic-units/doors/
```

```
src/features/kitchen-editor/catalogs/data/basic-units/drawers/
```

```
src/features/kitchen-editor/catalogs/data/basic-units/panels/
```

Cabinets

```
catalogId: "cabinets"
```

Categories:

```
categoryId: "base-cabinets"
```

```
categoryId: "wall-cabinets"
```

```
categoryId: "pantry-cabinets"
```

Folders:

```
src/features/kitchen-editor/catalogs/data/cabinets/base-cabinets/
```

```
src/features/kitchen-editor/catalogs/data/cabinets/wall-cabinets/
```

```
src/features/kitchen-editor/catalogs/data/cabinets/pantry-cabinets/
```

The JSON field `catalogCategoryId` must match this pattern:

```
"<catalogId>/<categoryId>"
```

Example:

```
"catalogCategoryId": "cabinets/base-cabinets"
```

4. Required JSON structure

Every object JSON file must use this top-level shape:

```
{
  "id": "unique-object-id",
  "name": "Human Readable Name",
  "catalogCategoryId": "cabinets/base-cabinets",
  "defaultDistanceFromFloorInches": 0,
  "dimensions": {
    "widthInches": {},
    "depthInches": {},
    "heightInches": {}
  }
}
```

```
},  
"optionGroups": [],  
"components": []  
}
```

`defaultDistanceFromFloorInches` is optional, but should be used when a type normally starts above the floor.

Examples:

```
"defaultDistanceFromFloorInches": 0  
"defaultDistanceFromFloorInches": 54
```

For wall cabinets, use:

```
"defaultDistanceFromFloorInches": 54
```

For base and pantry cabinets, use:

```
"defaultDistanceFromFloorInches": 0
```

5. Dimensions

Each object must define:

```
widthInches  
depthInches  
heightInches
```

Use this format:

```
"widthInches": {  
  "label": "Width",  
  "control": "select",  
  "defaultValueInches": 24,  
  "optionsInches": [  
    { "valueInches": 18, "label": "18 in" },  
    { "valueInches": 24, "label": "24 in" },  
    { "valueInches": 30, "label": "30 in" }  
  ],  
  "allowCustomValue": true,  
  "minValueInches": 6,  
  "maxValueInches": 60,  
  "stepInches": 0.25  
}
```

Use `allowCustomValue: true` when the user should be able to choose `Custom...` and type a number.

The actual placed object stores only the final actual size:

```
configuration.sizeInches
```

Do not store a separate custom/standard mode in the JSON.

6. Option groups

Use `optionGroups` for property panel fields.

Example:

```

"optionGroups": [
  {
    "id": "appearance",
    "label": "Appearance",
    "options": [
      {
        "id": "cabinet-color",
        "label": "Cabinet color",
        "valueType": "string",
        "control": "select",
        "defaultValue": "#d8b27a",
        "choices": [
          { "value": "#d8b27a", "label": "Natural maple" },
          { "value": "#f8f4fc", "label": "White" },
          { "value": "#111827", "label": "Black" },
          { "value": "#9ca3af", "label": "Gray" }
        ]
      }
    ]
  }
]

```

Supported common option value types:

```

valueType: "string"
valueType: "number"
valueType: "boolean"

```

Supported common controls:

```

control: "select"
control: "number"
control: "checkbox"

```

For colors, currently use:

```

valueType: "string"
control: "select"

```

with hex color values.

7. Components

The `components` array defines what the object is made from.

Supported component kinds:

```

primitive-geometry
nested-assembly

```

7.1 Primitive geometry component

Use `primitive-geometry` for panels, rails, toe kicks, handles, fillers, and any direct box-shaped part.

Currently supported primitive geometry kind:

```

"geometry": { "kind": "box" }

```

Do not use unsupported primitive geometry kinds such as sphere, cylinder, mesh, or line until the engine supports them.

Example:

```
{
  "kind": "primitive-geometry",
  "id": "left-side-panel",
  "label": "Left Side Panel",
  "geometry": { "kind": "box" },
  "localPositionInches": {
    "xInches": { "value": -11.625 },
    "yInches": { "value": 0 },
    "zInches": { "value": 0 }
  },
  "sizeInches": {
    "widthInches": { "value": 0.75 },
    "depthInches": { "ref": "size.depthInches" },
    "heightInches": { "ref": "size.heightInches" }
  },
  "material": {
    "colorOptionId": "cabinet-color"
  },
  "role": "side-panel"
}
```

7.2 Nested assembly component

Use `nested-assembly` when the object contains another object, such as a cabinet containing a door or drawer.

Example:

```
{
  "kind": "nested-assembly",
  "id": "door-front",
  "label": "Door Front",
  "definitionId": "door",
  "localPositionInches": {
    "xInches": { "value": 0 },
    "yInches": {
      "op": "add",
      "values": [
        {
          "op": "divide",
          "left": { "ref": "size.depthInches" },
          "right": { "value": 2 }
        },
        { "value": 0.375 }
      ]
    },
    "zInches": { "value": 0 }
  }
}
```

```

},
"configuration": {
  "sizeInches": {
    "widthInches": { "ref": "size.widthInches" },
    "depthInches": { "value": 0.75 },
    "heightInches": { "ref": "size.heightInches" }
  },
  "optionValues": {
    "door-color": { "ref": "option.door-color" },
    "handle-color": { "ref": "option.handle-color" },
    "show-handle": { "ref": "option.show-handles" },
    "handle-type": { "ref": "option.door-handle-type" },
    "handle-position": { "ref": "option.door-handle-position" }
  }
},
"role": "door-front"
}

```

`definitionId` must match the `id` of another JSON-defined assembly, such as:

```

door
drawer
panel

```

8. Component rotation

Components may define `localRotationDegrees`.

Example:

```

"localRotationDegrees": {
  "zDegrees": { "value": 90 }
}

```

Current practical rule:

- Use `zDegrees` only.
- Do not rely on `xDegrees` or `yDegrees` in object JSON until the renderer/build tree fully supports them.
- The current build tree applies Z rotation for nested assembly and primitive world rotation.

9. Expression system

JSON cannot use JavaScript formulas. Use structured expressions.

9.1 Literal value

```

{ "value": 24 }
{ "value": "#f8fafc" }
{ "value": true }

```

9.2 Reference current object size

```

{ "ref": "size.widthInches" }
{ "ref": "size.depthInches" }

```

```
{ "ref": "size.heightInches" }
```

9.3 Reference option value

```
{ "ref": "option.cabinet-color" }
```

```
{ "ref": "option.show-handles" }
```

All option references must point to an option that exists in `optionGroups`.

9.4 Reference current component size

These are available after the component size is evaluated:

```
{ "ref": "component.size.widthInches" }
```

```
{ "ref": "component.size.depthInches" }
```

```
{ "ref": "component.size.heightInches" }
```

Use component-size references when local position depends on the component's own calculated size.

10. Supported math operators

Add

```
{  
  "op": "add",  
  "values": [  
    { "ref": "size.widthInches" },  
    { "value": 2 }  
  ]  
}
```

Subtract

```
{  
  "op": "subtract",  
  "left": { "ref": "size.widthInches" },  
  "right": { "value": 1.5 }  
}
```

Multiply

```
{  
  "op": "multiply",  
  "left": { "ref": "size.widthInches" },  
  "right": { "value": 0.5 }  
}
```

Divide

```
{  
  "op": "divide",  
  "left": { "ref": "size.widthInches" },  
  "right": { "value": 2 }  
}
```

Do not divide by zero.

Min

```
{
  "op": "min",
  "values": [
    { "value": 5 },
    { "ref": "size.heightInches" }
  ]
}
```

Max

```
{
  "op": "max",
  "values": [
    { "value": 0.5 },
    { "ref": "size.heightInches" }
  ]
}
```

Clamp

```
{
  "op": "clamp",
  "value": { "ref": "size.widthInches" },
  "min": { "value": 6 },
  "max": { "value": 60 }
}
```

Negate

```
{
  "op": "negate",
  "value": { "ref": "size.widthInches" }
}
```

Do not use string formulas such as:

```
"xInches": "width / 2 - 0.375"
```

Always use structured expressions.

11. Conditions

Use `includeWhen`` to show or hide a component.

Equals

```
"includeWhen": {
  "ref": "option.show-handle",
  "equals": true
}
```

All

```
"includeWhen": {
  "all": [
```



```
{ "ref": "option.show-handle", "equals": true },
{ "ref": "option.handle-type", "equals": "bar" }
]
}
```

Any

```
"includeWhen": {
  "any": [
    { "ref": "option.handle-position", "equals": "left-center" },
    { "ref": "option.handle-position", "equals": "right-center" }
  ]
}
```

Not

```
"includeWhen": {
  "not": {
    "ref": "option.show-handle",
    "equals": false
  }
}
```

12. Coordinate rules

The object local coordinate system is:

```
X = width direction
Y = depth direction
Z = height direction
```

At `rotationDegrees.zDegrees = 0`:

```
front = +Y
back = -Y
left/right = X
up/down = Z
```

For doors, drawers, cabinets, and boxes:

- Front faces should sit outside the box body on `+Y`.
- Handles should protrude farther outward in `+Y`.
- Door/drawer fronts should be outside the cabinet body, not embedded inside the box structure.

For a front mounted on the cabinet front:

```
front center y = cabinet depth / 2 + front depth / 2
```

Example:

```
"yInches": {
  "op": "add",
  "values": [
    {
      "op": "divide",
      "left": { "ref": "size.depthInches" },
      "right": { "value": 2 }
    }
  ]
}
```

```
},  
  { "value": 0.375 }  
]  
}
```

Use inches for all object data. Do not use pixels in JSON object definitions.

13. Material rules

Use ``material.colorOptionId`` when the color should come from an option:

```
"material": {  
  "colorOptionId": "cabinet-color"  
}
```

Use ``material.colorHex`` only for fixed colors:

```
"material": {  
  "colorHex": "#111827"  
}
```

Optional opacity:

```
"material": {  
  "colorOptionId": "cabinet-color",  
  "opacity": { "value": 0.5 }  
}
```

If a primitive material has neither ``colorHex`` nor ``colorOptionId``, the engine uses its default primitive material color. Prefer explicit material definitions for authored catalog objects.

14. Component overrides are not raw object definition JSON

The placed assembly configuration may support ``componentOverrides`` for hiding or recoloring built component paths.

Do not put ``componentOverrides`` inside raw catalog object definition JSON.

``componentOverrides`` belong to placed/exported assembly instance data only. They are not part of the object creation guideline for reusable raw definitions.

15. Cabinet construction rules

Cabinets should be built from generic components, not cabinet-specific code.

A cabinet normally includes:

- left side panel
- right side panel
- back panel
- top panel
- bottom panel
- toe kick for base/pantry cabinets

- horizontal front rails when the front layout has stacked rows, including drawer/drawer splits and drawer/door splits
- vertical rails when a multi-column layout needs them
- nested door assemblies
- nested drawer assemblies

A one-door cabinet should not include an unnecessary front rail.

Door/drawer fronts may visually cover internal rails where appropriate, but internal rails should not overlap structural panels or other rails.

15.1 Opening zone rule

When creating or editing cabinet JSON, cabinet rails must be bounded by the opening zone they belong to.

A rail belongs to an opening zone. The rail size and position must be calculated from that opening zone.

Do not size rails from the full cabinet height unless the full cabinet height is truly the opening zone.

Bad:

```
vertical rail height = full cabinet height
vertical rail z center = 0
```

Good:

```
vertical rail bottom = opening bottom
vertical rail top = opening top
vertical rail height = opening top - opening bottom
vertical rail center = (opening bottom + opening top) / 2
```

Rails should touch surrounding panels or other rails by edge contact only. They should not overlap through panels, top/bottom boards, side panels, horizontal rails, or other vertical rails.

15.2 Cabinet opening zone formulas

For a cabinet box, define the usable front opening inside the structural panels.

For base and pantry cabinets with toe kick:

```
cabinetBottomZ = -heightInches / 2
cabinetTopZ = heightInches / 2
boxBottomZ = cabinetBottomZ + toeKickHeightInches
boxTopZ = cabinetTopZ
openingBottomZ = boxBottomZ + bottomPanelThicknessInches
openingTopZ = boxTopZ - topPanelThicknessInches
```

For wall cabinets with no toe kick:

```
boxBottomZ = -heightInches / 2
boxTopZ = heightInches / 2
openingBottomZ = boxBottomZ + bottomPanelThicknessInches
openingTopZ = boxTopZ - topPanelThicknessInches
```

Any rail inside the front opening should fit between these opening bounds unless a smaller sub-zone applies.

15.3 Two-door vertical rail rule

For a two-door cabinet with only doors, the vertical center rail should fit between the top and bottom panels.

```
verticalRailBottomZ = openingBottomZ
verticalRailTopZ = openingTopZ
verticalRailHeight = openingTopZ - openingBottomZ
verticalRailCenterZ = (openingBottomZ + openingTopZ) / 2
```

The vertical rail should touch:

- bottom panel top face
- top panel bottom face

It should not overlap:

- bottom panel
- top panel
- side panels

15.4 Drawer/door horizontal front rail rule

For a cabinet with drawer fronts above door fronts, the horizontal rail represents the split line between drawer and door zones.

```
horizontalRailCenterZ = drawer/door split line
horizontalRailBottomZ = horizontalRailCenterZ - frontRailHeightInches / 2
horizontalRailTopZ = horizontalRailCenterZ + frontRailHeightInches / 2
```

The horizontal rail width should be bounded between the side panels:

```
horizontalRailWidth = cabinetWidthInches - sidePanelThicknessInches * 2
horizontalRailCenterX = 0
```

The horizontal rail should touch the side panel inner faces by edge contact only. It should not overlap the side panels.

For a drawer + door cabinet:

- drawer bottom aligns with the front rail center
- door top aligns with the front rail center
- door extends to the bottom of the cabinet box body
- front rail is internal/behind fronts

15.5 Stacked drawer horizontal front rail rule

For a cabinet with multiple stacked drawer fronts, add one horizontal front rail between each pair of adjacent drawer rows.

Rail count:

```
drawerRowCount = number of stacked drawer rows
horizontalFrontRailCount = drawerRowCount - 1
```

Examples:

```
2 stacked drawers = 1 horizontal front rail
3 stacked drawers = 2 horizontal front rails
4 stacked drawers = 3 horizontal front rails
```

Each horizontal front rail belongs to the split line between two drawer opening zones.

```
horizontalRailCenterZ = drawer split line
horizontalRailBottomZ = horizontalRailCenterZ - frontRailHeightInches / 2
horizontalRailTopZ = horizontalRailCenterZ + frontRailHeightInches / 2
```

The rail width must stay inside the side panels:

```
horizontalRailWidth = cabinetWidthInches - sidePanelThicknessInches * 2  
horizontalRailCenterX = 0
```

The drawer horizontal front rail should touch the left/right side panel inner faces by edge contact only.

It should not overlap:

- left side panel
- right side panel
- top panel
- bottom panel
- another horizontal rail
- vertical rails

Drawer fronts may visually cover the rail from the front, but the rail should remain an internal structural component behind the drawer fronts.

Generalized rule:

```
horizontal front rail count = number of vertical front rows - 1
```

This applies to drawer-only layouts and mixed drawer/door layouts.

15.6 Lower vertical rail under a drawer/door split

For a cabinet with a horizontal rail and two lower doors, the lower vertical rail should live only in the lower door opening zone.

```
lowerVerticalRailBottomZ = openingBottomZ  
lowerVerticalRailTopZ = horizontalRailBottomZ  
lowerVerticalRailHeight = lowerVerticalRailTopZ - lowerVerticalRailBottomZ  
lowerVerticalRailCenterZ = (lowerVerticalRailBottomZ + lowerVerticalRailTopZ) / 2
```

The lower vertical rail should touch:

- bottom panel top face
- horizontal rail bottom face

It should not overlap:

- bottom panel
- horizontal rail
- top panel

15.7 Upper vertical rail above a drawer/door split

For cabinets with two upper drawers and two lower doors, the upper vertical rail should live only in the upper drawer opening zone.

```
upperVerticalRailBottomZ = horizontalRailTopZ  
upperVerticalRailTopZ = openingTopZ  
upperVerticalRailHeight = upperVerticalRailTopZ - upperVerticalRailBottomZ  
upperVerticalRailCenterZ = (upperVerticalRailBottomZ + upperVerticalRailTopZ) / 2
```

The upper vertical rail should touch:

- horizontal rail top face
- top panel bottom face

It should not overlap:

- horizontal rail
- top panel
- bottom panel

15.8 Rail overlap rules

Do not let these parts overlap unless it is intentionally decorative and clearly explained:

- vertical rail with bottom panel
- vertical rail with top panel
- vertical rail with horizontal rail
- vertical rail with another vertical rail
- horizontal rail with side panels
- horizontal rail with top/bottom panels
- front rail with door/drawer front if the rail is meant to be internal behind the front

Internal rails can sit behind door/drawer fronts in the Y direction, but they should not overlap other internal structural rails or box panels.

Final rail rule:

Cabinet rails should be sized by the opening zone they divide.
Rails should touch panels/rails by edge contact, not overlap volumes.

16. Door/drawer front visibility options for cabinet inspection

For cabinet JSON, add front visibility options so rails and panels can be inspected.

Cabinets with doors should include:

```
{
  "id": "show-doors",
  "label": "Show doors",
  "valueType": "boolean",
  "control": "checkbox",
  "defaultValue": true
}
```

Cabinets with drawers should include:

```
{
  "id": "show-drawers",
  "label": "Show drawers",
  "valueType": "boolean",
  "control": "checkbox",
  "defaultValue": true
}
```

Put these in an option group:

```
{
  "id": "front-visibility",
  "label": "Front Visibility",
  "options": []
}
```

```
}
```

Apply `includeWhen` only to nested door and drawer fronts.

Door components:

```
"includeWhen": {  
  "ref": "option.show-doors",  
  "equals": true  
}
```

Drawer components:

```
"includeWhen": {  
  "ref": "option.show-drawers",  
  "equals": true  
}
```

Do not apply these visibility conditions to:

- side panels
- back panel
- top panel
- bottom panel
- toe kick
- horizontal rail
- vertical rail
- fixed internal structure

The point of these options is to hide the fronts while keeping the cabinet structure visible for inspection.

17. Adding a new cabinet JSON file

Example path:

```
src/features/kitchen-editor/catalogs/data/cabinets/base-cabinets/base-three-door-cabinet.json
```

Required top fields:

```
{  
  "id": "base-three-door-cabinet",  
  "name": "Base Cabinet - 3 Doors",  
  "catalogCategoryId": "cabinets/base-cabinets",  
  "defaultDistanceFromFloorInches": 0,  
  "dimensions": {},  
  "optionGroups": [],  
  "components": []  
}
```

Then update:

```
src/features/kitchen-editor/catalogs/registry/kitchenEditorRawCatalogEntries.ts
```

Add the import:

```
import baseThreeDoorCabinetRawDefinition from "../data/cabinets/base-cabinets/base-three-door-cabinet.json";
```

Then add the registry entry:

```
{
```

```
catalogId: "cabinets",
categoryId: "base-cabinets",
rawDefinition: baseThreeDoorCabinetRawDefinition,
}
```

18. Best way to create a new object

Start by copying the closest existing JSON file.

Examples:

- For a new door, copy `data/basic-units/doors/door.json`.
- For a new drawer, copy `data/basic-units/drawers/drawer.json`.
- For a new panel, copy `data/basic-units/panels/panel.json`.
- For a one-drawer/one-door box, copy `data/basic-units/boxes/one-drawer-one-door-box.json`.
- For a one-door base cabinet, copy `data/cabinets/base-cabinets/base-one-door-cabinet.json`.
- For a drawer/door cabinet, copy `data/cabinets/base-cabinets/base-one-door-one-drawer-cabinet.json`.
- For a multi-drawer cabinet, copy the closest drawer cabinet such as `base-two-drawer-cabinet.json`, `base-three-drawer-cabinet.json`, or `base-four-drawer-cabinet.json`.
- For a wall cabinet, copy `data/cabinets/wall-cabinets/wall-one-door-cabinet.json`.

Then change:

- `id`
- `name`
- `catalogCategoryId` if needed
- dimension defaults/options
- option defaults
- components
- component positions/sizes
- registry import and entry

Keep object build data in JSON. Only use TypeScript changes for new generic engine capabilities.

19. General JSON quality checklist

Before returning a JSON file, check:

- JSON is valid.
- `id` is unique.
- File location matches the intended catalog/category.
- `catalogCategoryId` matches the folder/category and uses `/`.
- The JSON file is imported in `catalogs/registry/kitchenEditorRawCatalogEntries.ts`.
- The registry entry uses the correct `catalogId`, `categoryId`, and `rawDefinition`.
- Dimensions include `widthInches`, `depthInches`, and `heightInches`.
- Dimension defaults are inside the allowed option/min/max range.
- All option references exist in `optionGroups`.
- All nested `definitionId` values exist in the raw catalog registry.

- All components have stable unique IDs within the definition.
- Only inches fields are used for object data.
- No pixels are used in JSON object definitions.
- No JavaScript formula strings are used.
- Structured expressions are used for all calculations.
- `primitive-geometry` uses only `{ "kind": "box" }` unless the engine is expanded.
- Component rotation uses only `localRotationDegrees.zDegrees` for now.
- No object-specific TypeScript builder was created.
- No legacy/fallback/compat behavior was added.
- No fake aliases, fake wrappers, or old-name re-exports were added.
- Unsupported requested behavior is identified as a missing generic engine capability instead of being forced into invalid JSON.

20. Cabinet JSON validation checklist

Before returning cabinet JSON, check:

- One-door cabinets do not include unnecessary front rails.
- Two-door cabinets have vertical rails bounded between top/bottom panels.
- Drawer + door cabinets have horizontal rails at the drawer/door split.
- Two stacked drawers include exactly 1 horizontal front rail.
- Three stacked drawers include exactly 2 horizontal front rails.
- N stacked drawer rows include N - 1 horizontal front rails.
- Drawer horizontal front rails are bounded between side panels.
- Drawer horizontal front rails touch side panel inner faces by edge contact only.
- Drawer horizontal front rails do not overlap side panels, top/bottom panels, vertical rails, or other horizontal rails.
- Lower vertical rails stop below the horizontal rail.
- Upper vertical rails start above the horizontal rail.
- Vertical rails do not overlap bottom panels.
- Vertical rails do not overlap top panels.
- Vertical rails do not overlap horizontal rails.
- Horizontal rails do not overlap side panels.
- Door/drawer fronts may cover rails visually, but rails should not overlap other structural rails/panels.
- Show doors/show drawers options hide only fronts, not rails or panels.
- Rail formulas are based on opening-zone bounds, not full cabinet height.
- Front faces are outside the body on +Y.
- Handles protrude farther outward in +Y than fronts.
- Toe kick is included for base/pantry cabinets where appropriate.
- Wall cabinets use default distance from floor of 54 inches unless intentionally different.
- Base and pantry cabinets use default distance from floor of 0 inches unless intentionally different.

21. Final rule for AI assistants

When asked to create or edit a Kitchen Editor object JSON definition:

4. Use the closest existing JSON file as the starting point.
5. Keep the object definition in JSON.
6. Use the universal raw assembly format only.
7. Use structured expressions only.
8. Keep all model data in inches.
9. Place fronts and handles on +Y.
10. For cabinets, calculate rails from the opening zones they divide.
11. For stacked drawer rows, create one horizontal front rail between each adjacent drawer row.
12. Register the JSON in `catalogs/registry/kitchenEditorRawCatalogEntries.ts`.
13. Do not invent unsupported JSON fields, object-specific builders, fallbacks, wrappers, or compatibility shims.